University of Alicante

Computer Engineering – Specialisation in Computing

VAR: Computer Vision and Robotics

3D Gaussian Splatting

for Real-time Rendering and SLAM

A Comparative Study with Neural Radiance Fields

Authors: Jaime Hernández and Yauheni Yatsevivh

Date: March 2026

Contents

1 Introduction **3**
- 1.1 Novel View Synthesis . . . 3
- 1.2 Neural Radiance Fields (NeRF) . . . 3
- 1.3 3D Gaussian Splatting (3DGS) . . . 3
- 1.4 Comparative Analysis: NeRF vs. 3DGS . . . 4

2 Mathematical Representation of 3D Gaussians **5**
- 2.1 The 3D Gaussian Kernel . . . 5
- 2.2 Gaussian Primitive . . . 5
- 2.3 Scene Representation and Parameter Count . . . 5

3 Covariance Factorisation and Rotation **6**
- 3.1 The RS Factorisation . . . 6
- 3.2 Quaternion Representation of Rotation . . . 6

4 Colour Modelling via Spherical Harmonics **7**
- 4.1 Motivation . . . 7
- 4.2 View-Dependent Colour . . . 7
- 4.3 Degree Selection . . . 7

5 Alpha Blending and Accumulation Rendering **8**
- 5.1 Alpha Compositing . . . 8
- 5.2 Per-Gaussian Opacity at a Pixel . . . 8
- 5.3 Incremental Accumulation . . . 8
- 5.4 Connection to NeRF Volume Rendering . . . 8

6 Optimisation Mechanisms: Densification, Pruning, and Opacity **9**
- 6.1 Training Objective . . . 9
- 6.2 Optimiser . . . 9
- 6.3 Adaptive Density Control . . . 9
- 6.4 Pruning . . . 9
- 6.5 Periodic Opacity Reset . . . 9

7 3D Gaussian Splatting for SLAM **10**
- 7.1 Simultaneous Localisation and Mapping . . . 10
- 7.2 Integration of 3DGS into the SLAM Pipeline . . . 10
- 7.3 Keyframe Management and Memory Control . . . 11
- 7.4 Notable 3DGS-SLAM Systems . . . 11
- 7.5 Advantages and Open Challenges . . . 11

8 Uncertainty Management and Point Initialization **12**

9 Loop Closure and Global Optimization **12**

10 Performance Analysis: Latency, FPS, and VRAM Consumption **13**
- 10.1 Computational Latency and FPS . . . 13
- 10.2 VRAM Consumption and Scalability . . . 13
- 10.3 SLAM Evaluation Metrics . . . 14

11 Limitations: Dynamic Scenes, Compression, and Storage **15**

12 Conclusions and Future Directions **16**

1 Introduction

1.1 Novel View Synthesis

Novel view synthesis (NVS) aims to generate photorealistic images from arbitrary viewpoints given a set of images captured from known camera positions. Applications include virtual reality, autonomous driving, and digital heritage preservation. Deep learning has replaced traditional multi-view stereo methods with learned representations, and two have emerged as state-of-the-art: **NeRF** [3] and **3DGS** [2].

1.2 Neural Radiance Fields (NeRF)

NeRF represents a scene as a continuous volumetric function:

$$F_\theta : (\mathbf{x}, \mathbf{d}) \longrightarrow (\mathbf{c}, \sigma) \tag{1}$$

where F_θ is an MLP mapping position $\mathbf{x} \in \mathbb{R}^3$ and viewing direction $\mathbf{d}$ to colour $\mathbf{c}$ and density σ. A pixel colour is computed by integrating along the ray:

$$\hat{C}(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\,\sigma(\mathbf{r}(t))\,\mathbf{c}(\mathbf{r}(t), \mathbf{d})\,\mathrm{d}t, \quad T(t) = \exp\left(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))\,\mathrm{d}s\right) \tag{2}$$

Limitations. NeRF requires 1–2 days of training and renders at 5–30 s/frame due to the cost of per-pixel MLP evaluations.

1.3 3D Gaussian Splatting (3DGS)

3DGS represents the scene as a set of *explicit, learnable 3D Gaussian primitives.* Each Gaussian $\mathcal{G}_i$ stores a position $\boldsymbol{\mu}_i \in \mathbb{R}^3$, a covariance $\boldsymbol{\Sigma}_i$, an opacity o_i, and view-dependent colour as spherical harmonic (SH) coefficients. Rendering is performed by projecting each 3D Gaussian onto the image plane (splatting) and compositing the resulting 2D splats via differentiable alpha blending on the GPU, achieving >90 fps at 1080p. Gaussians are initialised from a SfM sparse point cloud and optimised end-to-end with adaptive density control (Section 6).

1.4 Comparative Analysis: NeRF vs. 3DGS

Table 1: Qualitative comparison between NeRF and 3DGS.

Criterion	**NeRF**	**3D Gaussian Splatting**
Representation	Implicit (MLP)	Explicit (Gaussian primitives)
Rendering method	Ray casting + volume integration	GPU rasterisation (splatting)
Render speed	<1–30 fps	>90 fps (consumer GPU)
Training time	1–2 days; 5 min (Instant-NGP)	20–40 minutes
Memory (inference)	Low (MLP weights)	High (∼1 GB/scene)
Editability	Difficult	Easy (explicit geometry)

NeRF's implicit representation handles smooth geometry naturally; 3DGS trades spatial continuity for hardware-accelerated rendering and explicit editability.

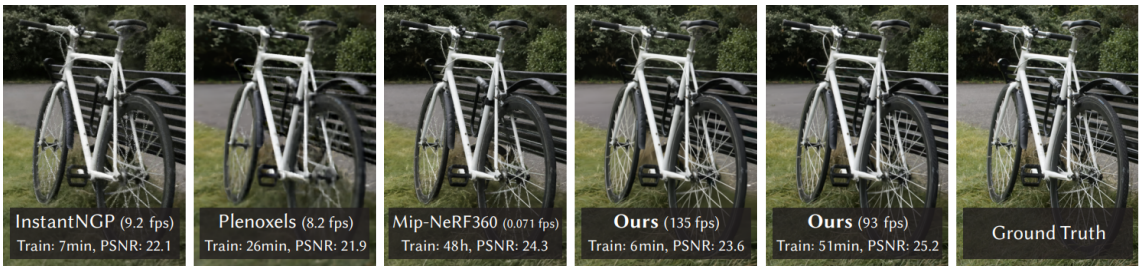

Figure 1: Photo-realistic novel views synthesised with 3D Gaussian Splatting at >90 fps on a consumer GPU. Each scene is reconstructed from a set of posed photographs; the Gaussian map supports real-time rendering from any viewpoint. Image from Kerbl et al. [2].

The remainder of this document covers the mathematical foundations of 3DGS: geometric representation (Sections 2–4), the rendering pipeline (Section 5), the optimisation strategy (Section 6), and the extension to SLAM (Section 7).

2 Mathematical Representation of 3D Gaussians

2.1 The 3D Gaussian Kernel

A 3D Gaussian is defined by its unnormalised influence kernel:

$$\boxed{\mathcal{G}(\mathbf{x}\,;\,\boldsymbol{\mu},\boldsymbol{\Sigma}) = \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)} \tag{3}$$

where $\boldsymbol{\mu} \in \mathbb{R}^3$ is the centre and $\boldsymbol{\Sigma} \in \mathbb{R}^{3\times 3}$ is a symmetric positive semi-definite (SPSD) covariance matrix. The kernel equals 1 at $\mathbf{x} = \boldsymbol{\mu}$ and decays with the Mahalanobis distance; the locus $\mathcal{G} = e^{-1/2}$ traces the *one-sigma ellipsoid*.

2.2 Gaussian Primitive

Gaussian Primitive

Each primitive $\mathcal{G}_i$ is an oriented, anisotropic ellipsoid characterised by:

- $\boldsymbol{\mu}_i \in \mathbb{R}^3$ – **position**,
- $\boldsymbol{\Sigma}_i \in \mathbb{R}^{3\times 3}$ – **shape and orientation** (eigenvalues = squared semi-axis lengths; eigenvectors = principal axes),
- $o_i \in [0,1]$ – **opacity**,
- $\mathbf{f}_i \in \mathbb{R}^{(L+1)^2\times 3}$ – **colour** as SH coefficients (Section 4).

2.3 Scene Representation and Parameter Count

A scene is a finite mixture of N primitives:

$$\mathcal{S} = \left\{(\boldsymbol{\mu}_i,\, \boldsymbol{\Sigma}_i,\, o_i,\, \mathbf{f}_i)\right\}_{i=1}^{N} \tag{4}$$

Typical scenes contain $N \in [10^5, 10^7]$ Gaussians stored as flat GPU tensors. For SH degree $L = 3$ each Gaussian has 59 learnable parameters $(3+4+3+1+48)$, so 3×10^6 Gaussians require ≈ 700 MB in `float32`.

3 Covariance Factorisation and Rotation

3.1 The RS Factorisation

To guarantee SPSD throughout optimisation, 3DGS parametrises $\mathbf{\Sigma}$ as:

$$\boxed{\mathbf{\Sigma} = \mathbf{R}\,\mathbf{S}\,\mathbf{S}^\top\mathbf{R}^\top} \tag{5}$$

where $\mathbf{R} \in SO(3)$ is a rotation matrix and $\mathbf{S} = \mathrm{diag}(s_1, s_2, s_3)$ with $s_k > 0$. SPSD is guaranteed since $\mathbf{v}^\top\mathbf{\Sigma}\,\mathbf{v} = \|\mathbf{S}^\top\mathbf{R}^\top\mathbf{v}\|^2 \geq 0$ for all $\mathbf{v}$. Expanded:

$$\mathbf{\Sigma} = \mathbf{R}\begin{pmatrix} s_1^2 & 0 & 0 \\ 0 & s_2^2 & 0 \\ 0 & 0 & s_3^2 \end{pmatrix}\mathbf{R}^\top \tag{6}$$

The columns of $\mathbf{R}$ are the eigenvectors of $\mathbf{\Sigma}$ and s_k^2 are the eigenvalues, so s_k are the principal semi-axis lengths.

3.2 Quaternion Representation of Rotation

$\mathbf{R}$ is stored as a unit quaternion $\mathbf{q} = (q_w, q_x, q_y, q_z)$, $\|\mathbf{q}\| = 1$, which uses only 4 parameters and avoids gimbal lock:

$$\mathbf{R}(\mathbf{q}) = \begin{pmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & 1 - 2(q_x^2 + q_y^2) \end{pmatrix} \tag{7}$$

After each gradient step $\mathbf{q}$ is renormalised: $\mathbf{q} \leftarrow \mathbf{q}/\|\mathbf{q}\|$.

4 Colour Modelling via Spherical Harmonics

4.1 Motivation

Surfaces exhibit view-dependent appearance (specular highlights, metallic reflections). In 3DGS each Gaussian encodes its view-dependent colour compactly using **spherical harmonics (SH)**, an orthonormal basis on the unit sphere S^2

4.2 View-Dependent Colour

For Gaussian $\mathcal{G}_i$ and channel $k \in \{R, G, B\}$, the colour in direction $\mathbf{d}$ is:

$$\boxed{c_{i,k}(\mathbf{d}) = \sum_{l=0}^{L} \sum_{m=-l}^{l} f_{i,k,l}^{m} \, Y_l^m(\mathbf{d})} \tag{8}$$

where $f_{i,k,l}^{m}$ are learned coefficients. In matrix form: $\mathbf{c}_i(\mathbf{d}) = \mathbf{F}_i \, \mathbf{y}(\mathbf{d})$, with $\mathbf{y}(\mathbf{d}) \in \mathbb{R}^{(L+1)^2}$ the vector of SH values. SH evaluation is purely polynomial, making it very efficient on the GPU.

4.3 Degree Selection

Table 2: SH parameter count by degree.

Max Degree L	Coeffs/channel	Total (RGB)	Effect
0	1	3	Constant diffuse
1	4	12	Linear shading
2	9	27	Quadratic specular
3	16	48	Sharp specular (used in 3DGS)

The original 3DGS paper uses $L = 3$. Training starts at $L = 0$ and unlocks one degree every 1000 iterations, stabilising early optimisation by fitting low-frequency geometry first.

5 Alpha Blending and Accumulation Rendering

5.1 Alpha Compositing

Given N colour contributions $\{c_i\}$ with opacities $\{\alpha_i\}$ sorted front-to-back, the final pixel colour is:

$$\boxed{C = \sum_{i=1}^{N} c_i \, \alpha_i \, T_i}, \qquad T_i = \prod_{j=1}^{i-1}(1-\alpha_j), \quad T_1 = 1 \tag{9}$$

T_i is the transmittance reaching layer i; $\alpha_i T_i$ is its effective contribution.

5.2 Per-Gaussian Opacity at a Pixel

The opacity of Gaussian $\mathcal{G}_i$ at pixel $\mathbf{u}$ combines the learned opacity o_i and the 2D kernel response:

$$\boxed{\alpha_i(\mathbf{u}) = o_i \cdot \exp\!\big(-\tfrac{1}{2}(\mathbf{u}-\boldsymbol{\mu}_i')^\top (\boldsymbol{\Sigma}_i')^{-1}(\mathbf{u}-\boldsymbol{\mu}_i')\big)} \tag{10}$$

5.3 Incremental Accumulation

Algorithm 1 Alpha Blending per Pixel (front-to-back)

Require: Sorted Gaussians $\{i_1, \ldots, i_K\}$ covering pixel $\mathbf{u}$
Ensure: Pixel colour $C(\mathbf{u})$

```
C ← 0,   T ← 1
for k = 1, ..., K do
    α ← α_{i_k}(u)
    if α < ε_min then continue
    end if
    C ← C + c_{i_k} · α · T;   T ← T · (1 − α)
    if T < ε_stop then break
    end if
end for
return C + T · C_bg
```

$\epsilon_{\text{min}} = \epsilon_{\text{stop}} = 10^{-4}$. Depth and opacity maps are obtained by the same accumulation replacing $\mathbf{c}_i$ with z_i or 1:

$$D(\mathbf{u}) = \sum_i z_i \, \alpha_i \, T_i, \qquad A(\mathbf{u}) = 1 - T_{\text{final}} \tag{11}$$

5.4 Connection to NeRF Volume Rendering

The 3DGS formula is the discrete equivalent of the NeRF volume integral. Identifying $\alpha_i = 1 - e^{-\sigma_i \delta_i}$ and $T_i = \prod_{j<i} e^{-\sigma_j \delta_j}$ recovers Eq. (9) exactly. The two methods differ only in *how* α_i is produced, not in the compositing formula.

6 Optimisation Mechanisms: Densification, Pruning, and Opacity

6.1 Training Objective

All parameters are optimised end-to-end by minimising:

$$\boxed{\mathcal{L} = (1-\lambda)\,\mathcal{L}_1 + \lambda\,\mathcal{L}_{\text{D-SSIM}}}, \quad \lambda = 0.2 \tag{12}$$

$\mathcal{L}_1$ ensures per-pixel accuracy; $\mathcal{L}_{\text{D-SSIM}}$ preserves structural detail by penalising blurry renderings.

6.2 Optimiser

Adam with per-group learning rates (position: 1.6×10^{-4} decaying to 1.6×10^{-6}; scale: 5×10^{-3}; rotation: 10^{-3}; opacity: 5×10^{-2}; SH degree 0: 2.5×10^{-3}; SH 1–3: 2.5×10^{-4}).

6.3 Adaptive Density Control

Every 100 iterations (from iter. 500 to 15 000), the mean position gradient

$$\bar{\nabla}_i = \tfrac{1}{K} \sum_{k=1}^{K} \|\nabla_{\boldsymbol{\mu}_i'} \mathcal{L}\| \tag{13}$$

drives two operations when $\bar{\nabla}_i > \tau_{\text{grad}} = 2 \times 10^{-4}$:

- **Clone** (small Gaussian, $\max s_k < \tau_{\text{size}}$): create an exact copy; both are optimised independently. The region needs more primitives.
- **Split** (large Gaussian, $\max s_k \geq \tau_{\text{size}}$): replace with two Gaussians at scale $s_k/1.6$, positions sampled from $\mathcal{N}(\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$. The primitive covers too large an area.

6.4 Pruning

At the same frequency, Gaussians are removed if: (1) opacity $o_i < \epsilon_\alpha = 0.005$, (2) world-space size exceeds τ_w, or (3) projected 2D radius exceeds τ_v (removes "floaters").

6.5 Periodic Opacity Reset

Every 3000 iterations all opacities are reset to $\alpha_{\min} \approx 0.01$. Gaussians that contribute to the scene recover high opacity quickly; redundant ones do not and are pruned at the next pruning step. This prevents ghost Gaussians from accumulating throughout training.

7 3D Gaussian Splatting for SLAM

7.1 Simultaneous Localisation and Mapping

Simultaneous Localisation and Mapping (SLAM) addresses the chicken-and-egg problem of robotic perception: a moving agent must estimate its own pose (localisation) while concurrently building a map of the environment (mapping), without prior knowledge of either. Classical SLAM pipelines rely on sparse or semi-dense geometric representations such as point clouds, occupancy grids, and keyframe-based pose graphs. These representations are efficient but lack visual fidelity, making them unsuitable for applications that require photorealistic reconstructions, such as augmented reality or human-robot interaction.

3DGS offers a compelling alternative: the same Gaussian primitives used for novel-view synthesis can serve as the scene map, providing simultaneously a dense, photorealistic, and differentiable representation that can be queried or rendered in real time.

7.2 Integration of 3DGS into the SLAM Pipeline

A 3DGS-SLAM system alternates between two tightly coupled phases:

1. **Tracking.** Given the current Gaussian map, the camera pose $T_t \in SE(3)$ is estimated by minimising a photometric loss between the rendered frame (obtained by splatting the existing Gaussians from the candidate pose) and the actual camera observation:

$$T_t^* = \arg\min_{T_t} \sum_p \left\| \hat{I}(p;\, T_t,\, \mathcal{G}) - I_t(p) \right\|_1, \tag{14}$$

   where $\hat{I}(p;\, T_t,\, \mathcal{G})$ is the rendered colour at pixel p given map $\mathcal{G}$ and pose T_t, and I_t is the observed frame.

2. **Mapping.** Once the pose is fixed, the Gaussian map is updated by (a) spawning new Gaussians at the depth of pixels with high photometric error, and (b) optimising all Gaussian parameters via back-propagation through the differentiable renderer, exactly as in the offline 3DGS pipeline (Section 6).

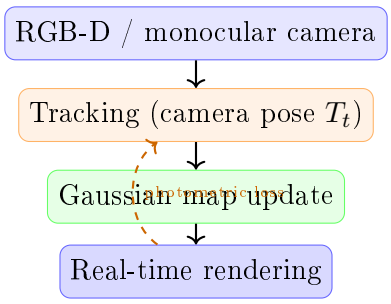

Figure 2: Data-flow in a 3DGS-SLAM system. Tracking and mapping alternate at each keyframe; the differentiable renderer couples both phases.

7.3 Keyframe Management and Memory Control

Unlike offline 3DGS, which optimizes over a static set of images, 3DGS-SLAM must manage a continuous stream of data. Processing every frame for mapping is computationally prohibitive; therefore, systems employ a **keyframing strategy**.

Keyframe Selection. A new frame I_t is designated as a keyframe based on two primary heuristics:

- **Geometric Displacement:** The camera has moved beyond a spatial threshold Δpos or rotated beyond $\Delta\theta$ relative to the last keyframe.
- **Photometric Overlap:** The current view is rendered using the existing map $\mathcal{G}$. If the intersection-over-union (IoU) of observed pixels or the structural similarity (SSIM) drops below a threshold, the scene is considered "new," triggering a keyframe.

Windowed Optimization. To maintain real-time performance, mapping optimization is typically restricted to a **sliding window** of recent keyframes. To prevent global drift, some architectures—such as SplaTAM [1]—periodically sample "global" keyframes from the entire trajectory history. This ensures that the Gaussian parameters remain consistent with previously visited areas of the scene, effectively acting as a local bundle adjustment (BA) equivalent.

7.4 Notable 3DGS-SLAM Systems

Table 3: Comparison of published 3DGS-SLAM methods.

System	Input	Key contribution	Speed
SplaTAM [1]	RGB-D	Silhouette-guided densification; first real-time 3DGS SLAM	∼10 fps mapping
Gaussian-SLAM	RGB-D	Online reconstruction with sub-map merging	>10 fps on one GPU
MonoGaussian	Monocular RGB	Depth estimated from monocular cues; no depth sensor	Real-time capable

7.5 Advantages and Open Challenges

Advantages over classical SLAM representations.

- **Photorealism.** The Gaussian map supports novel-view rendering, enabling downstream tasks such as loop closure via appearance-based place recognition.
- **Differentiability.** Pose optimisation is formulated as gradient descent through the renderer (Eq. 14), without requiring hand-crafted feature descriptors.

- **Real-time rendering.** The GPU-rasterised pipeline delivers >90 fps renders even during online mapping.

Open challenges.

- **Memory growth.** The number of Gaussians grows with scene size, making large-scale outdoor SLAM memory-intensive.
- **Loop closure.** Correcting accumulated drift requires updating potentially millions of Gaussians consistently.
- **Dynamic objects.** Moving people or vehicles violate the static scene assumption and produce rendering artefacts.
- **Monocular scale ambiguity.** Without depth, metric scale must be recovered from additional cues or a known object.

Summary

3DGS-SLAM unifies photorealistic scene reconstruction and real-time camera tracking in a single differentiable framework, combining the speed of GPU rasterisation with the accuracy of gradient-based pose optimisation. Current systems achieve real-time performance on RGB-D input; extending them to monocular and large-scale outdoor settings remains an active research area.

8 Uncertainty Management and Point Initialization

The transition from 2D observations to 3D Gaussian primitives requires robust initialization and filtering to maintain map integrity.

Depth-Based Seeding. In RGB-D configurations, the initial position μ of a Gaussian is back-projected from the depth map D_t. The initial scale s is typically set to:

$$s_{init} = \lambda \cdot \frac{D_t(p)}{f}, \tag{15}$$

where f is the focal length and λ is a scaling factor to ensure initial coverage.

Pruning and Densification. Uncertainty is managed through a "survive-of-the-fittest" approach. Gaussians are pruned if their opacity α falls below a threshold ϵ_α or if their view-space position exhibits high variance over multiple frames. Conversely, in regions where the rendered silhouette $S(p)$ is near zero but an observation exists, new Gaussians are spawned to fill "holes" in the geometry.

9 Loop Closure and Global Optimization

As the camera traverses large environments, small errors in pose estimation (Eq. 14) accumulate, leading to "drift" where the reconstructed map no longer aligns with reality upon revisiting a known location.

Place Recognition. Loop closure detection in 3DGS-SLAM typically leverages the photorealistic nature of the map. Systems compare the current live frame I_t against a database of previous keyframes using global image descriptors (e.g., NetVLAD or DBoW2). If a high-similarity match is found, a loop candidate is identified.

Global Bundle Adjustment (GBA). Once a loop is detected, the system must perform a global correction. Unlike traditional SLAM, which only optimizes a pose graph, 3DGS-SLAM must adjust:

- **Camera Poses:** Correcting the historical trajectory to close the geometric gap.
- **Gaussian Parameters:** Aligning the primitives $\{\mu_i, \Sigma_i\}$ to the new optimized poses.

Sub-map Merging. Advanced systems like *Gaussian-SLAM* manage large-scale scenes by dividing the environment into independent sub-maps. When a loop is closed, these sub-maps are merged by performing a rigid transformation on the Gaussian collections to ensure seamless visual and geometric transitions.

10 Performance Analysis: Latency, FPS, and VRAM Consumption

The fundamental architectural differences between NeRF (implicit, continuous) and 3DGS (explicit, discrete) manifest most prominently in their hardware utilization during SLAM operations. The choice between the two paradigms generally represents a trade-off between computational latency and memory consumption.

10.1 Computational Latency and FPS

NeRF-based SLAM systems, such as iMAP and NICE-SLAM, rely on volumetric ray marching. To render a single pixel for photometric loss calculation, the system must query a Multi-Layer Perceptron (MLP) or feature grid hundreds of times along a camera ray. This heavy computational burden restricts tracking and mapping speeds to typically 1–5 Frames Per Second (FPS), and makes real-time novel view rendering (> 30 FPS) nearly impossible without specialized hardware.

In contrast, 3DGS-SLAM systems (e.g., SplaTAM [1], Gaussian-SLAM) bypass ray marching in favor of tile-based rasterization. Because projecting 3D ellipsoids onto a 2D plane is a highly parallelizable and native GPU operation, tracking and mapping can be performed at 10–30 FPS. Furthermore, the decoupling of the mapping thread from the rendering thread allows 3DGS-SLAM to render the live reconstructed scene at > 90 FPS, making it vastly superior for real-time AR/VR applications.

10.2 VRAM Consumption and Scalability

While 3DGS excels in compute speed, it suffers from a severe memory bottleneck. NeRFs encode the scene implicitly; the entire geometry and appearance of a room

can be compressed into the weights of an MLP and a multi-resolution hash grid, often consuming less than 1 GB of Video RAM (VRAM).

Conversely, 3DGS requires explicit storage for every primitive. Each 3D Gaussian must store its position $\mu \in \mathbb{R}^3$, covariance matrix components (scale $s \in \mathbb{R}^3$ and rotation quaternion $q \in \mathbb{R}^4$), opacity $\alpha \in \mathbb{R}$, and Spherical Harmonic (SH) coefficients for view-dependent color. A single high-fidelity indoor room can easily require 1 to 3 million Gaussians, consuming 2–6 GB of VRAM. In large-scale trajectory mapping without aggressive pruning, the Gaussian count can explode, pushing VRAM consumption beyond 24 GB and severely limiting the deployment of 3DGS-SLAM on resource-constrained embedded systems or mobile robots.

Table 4: Empirical Performance Trade-offs in Real-Time SLAM

Metric	**NeRF-SLAM**	**3DGS-SLAM**
Tracking & Mapping Speed	1 – 5 FPS	10 – 30+ FPS
Rendering Speed	< 10 FPS	> 90 FPS
VRAM Footprint (Room Scale)	< 1 GB	2 – 6 GB
Primary Bottleneck	Computational (Compute Bound)	Memory (Memory Bound)
Mobile Suitability	Moderate (High Latency)	Poor (High VRAM)

10.3 SLAM Evaluation Metrics

To rigorously compare the performance of NeRF-based and 3DGS-SLAM systems, evaluations are typically divided into two distinct categories: tracking accuracy (localisation) and mapping fidelity (reconstruction).

Tracking Accuracy. The precision of the estimated camera trajectory is evaluated against a known ground-truth trajectory using the **Absolute Trajectory Error (ATE)**. The ATE measures the global consistency of the estimated trajectory by first aligning it to the ground truth (usually via Horn's method) and then computing the Root Mean Square Error (RMSE) of the translational differences. Mathematically, for an estimated trajectory T and ground truth Q, the ATE RMSE over N frames is defined as:

$$\mathrm{ATE}_{\mathrm{RMSE}} = \left(\frac{1}{N} \sum_{i=1}^{N} \left\| \mathrm{trans}(Q_i^{-1} S T_i) \right\|^2 \right)^{1/2} \tag{16}$$

where S is the rigid alignment transformation.

Mapping and Rendering Quality. Because both NeRF and 3DGS are inherently view-synthesis formulations, the quality of the reconstructed map is evaluated by freezing the map, rendering novel views from held-out ground-truth poses, and

comparing the results to the actual captured images. The standard photometric metrics include:

- **PSNR (Peak Signal-to-Noise Ratio):** Measures pixel-wise accuracy. Higher is better.
- **SSIM (Structural Similarity Index):** Measures perceived changes in structural information, capturing high-frequency details. Higher is better (maximum is 1.0).
- **LPIPS (Learned Perceptual Image Patch Similarity):** Evaluates deep-feature differences using a pre-trained network (like VGG), providing a metric that closely aligns with human visual perception. Lower is better.

While NeRF and 3DGS systems often achieve comparable ATE RMSE (typically in the range of 1 to 5 cm on standard datasets like Replica or TUM-RGBD), 3DGS systems generally edge out NeRFs in mapping metrics (PSNR, SSIM) due to the explicit representation preserving sharp high-frequency details better than continuous MLPs.

11 Limitations: Dynamic Scenes, Compression, and Storage

Despite the real-time rendering and tracking advantages of 3DGS over NeRF, the explicit nature of Gaussian splatting introduces unique limitations regarding scene dynamics and long-term data storage.

Handling Dynamic Scenes. Standard 3DGS-SLAM relies on a strict static-world assumption. When moving objects (e.g., pedestrians, vehicles, or moving shadows) enter the camera's view, the tracking thread registers high photometric error. The mapping thread responds by improperly spawning "floater" Gaussians in mid-air to satisfy the momentary view, resulting in persistent ghosting artefacts in the final map. While some NeRF-SLAM systems handle dynamics by learning transient density fields, 3DGS must adopt explicit temporal modelling. Recent extensions, such as *4D Gaussian Splatting (4DGS)* or *Dynamic 3DGS*, address this by attaching a time-dependent velocity vector or a learned deformation field to each Gaussian primitive. However, integrating these 4D representations into a real-time SLAM pipeline remains computationally prohibitive and is an active area of research.

Persistent Storage and Compression. While Section 10.2 discussed the run-time Video RAM (VRAM) bottleneck, the persistent storage (disk space) required to save a reconstructed Gaussian map is equally problematic. NeRF models can compress an entire scene into a few megabytes of neural network weights. In contrast, saving a 3DGS map involves serializing millions of primitives. A standard 3 million Gaussian scene requires approximately 700 MB to 1 GB of disk space in `float32` precision.

To mitigate this, recent literature has proposed aggressive compression techniques for explicit volumetric representations:

- **Vector Quantization (VQ):** Instead of storing unique Spherical Harmonic (SH) coefficients and covariance matrices for every Gaussian, primitives are clustered into a smaller "codebook" of shared properties.
- **Attribute Pruning:** The highest-order SH bands (degrees $L = 2$ and $L = 3$) consume the vast majority of parameters. Selectively dropping these bands for Gaussians in textureless regions drastically reduces the file size without degrading perceived quality.
- **Entropy Coding:** Sorting Gaussians spatially (e.g., via Morton coding) and applying standard lossless compression algorithms can reduce the storage footprint by over 50%, bringing the storage requirements closer to practical limits for mobile robotics.

12 Conclusions and Future Directions

En este estudio comparativo, hemos analizado los fundamentos matemáticos y la integración en sistemas SLAM de 3D Gaussian Splatting (3DGS) [**?**] frente a Neural Radiance Fields (NeRF) [3]. La transición de representaciones implícitas y continuas (NeRF) a representaciones explícitas y discretas (3DGS) marca un cambio de paradigma fundamental en los sistemas de percepción robótica, trasladando el principal cuello de botella del hardware desde la capacidad de cómputo hacia el consumo de memoria.

Por un lado, 3DGS ha demostrado ser abrumadoramente superior en términos de latencia y velocidad computacional. Al reemplazar el costoso proceso de *ray marching* volumétrico de NeRF por una rasterización de primitivas explícitas acelerada por GPU, 3DGS logra velocidades de seguimiento y mapeo de 10 a más de 30 FPS [1], junto con un renderizado fotorrealista en tiempo real que supera los 90 FPS. Además, su representación geométrica explícita preserva mejor los detalles de alta frecuencia en las métricas de evaluación de mapas (como PSNR y SSIM). Estas características hacen de 3DGS la tecnología idónea para aplicaciones interactivas como la realidad aumentada y virtual (AR/VR).

Sin embargo, estas ventajas conllevan compromisos críticos. La naturaleza explícita de 3DGS sufre de un severo consumo de memoria de video (VRAM) y espacio de almacenamiento persistente. Mientras que NeRF puede comprimir la información de una escena entera en menos de 1 GB de VRAM gracias a los pesos de su red neuronal, 3DGS requiere instanciar millones de elipsoides, pudiendo sobrepasar rápidamente los 24 GB en mapeos de trayectorias a gran escala. Adicionalmente, la dependencia de 3DGS de un entorno estrictamente estático hace que la presencia de objetos dinámicos genere artefactos visuales, requiriendo el uso de complejas extensiones temporales como 4DGS que hoy en día resultan computacionalmente prohibitivas.

En resumen, 3DGS-SLAM unifica la reconstrucción fotorrealista de alta fidelidad y el seguimiento de cámaras en tiempo real, superando las barreras de latencia de las arquitecturas NeRF. El éxito y la adopción futura de 3DGS en plataformas con recursos limitados (como robots móviles o sistemas integrados) dependerán de la investigación activa en técnicas agresivas de compresión, como la cuantización vectorial, la poda de atributos y la codificación de entropía, así como de un modelado

eficiente para escenas dinámicas.

References

[1] Nikhil Keetha, Jay Karhade, Krishna Murthy Jatavallabhula, Gengshan Yang, Sebastian Scherer, Deva Ramanan, and Jonathon Luiten. SplaTAM: Splat, track & map 3D Gaussians for dense RGB-D SLAM. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[2] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian Splatting for real-time radiance field rendering. In *ACM SIGGRAPH 2023 Conference Proceedings*. ACM, 2023.

[3] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, pages 405–421. Springer, 2020.